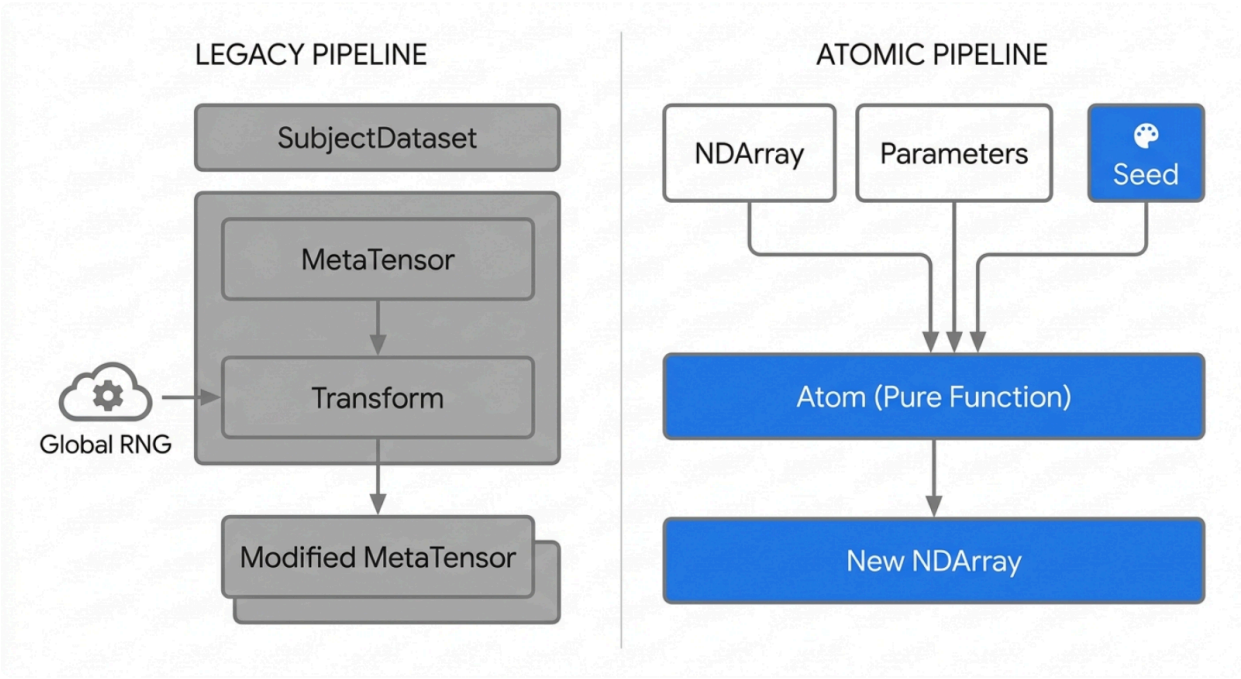
Engineering Pure-Function 3D Volumetric Augmentation Atoms for Medical and Tomographic Imaging

1. The Paradigm Shift to Functional Augmentation Architectures

The processing of highly complex volumetric data in medical imaging, structural biology, and computational tomography demands robust, computationally efficient, and strictly reproducible data augmentation pipelines. Historically, large-scale deep learning frameworks such as MONAI [1] and TorchIO [3] have dominated the landscape of three-dimensional medical image augmentation. While these extensive libraries provide broad and generalized functionality, they inherently encapsulate spatial and intensity operations within complex, stateful object-oriented class hierarchies. These include structures such as RandomizableTransform, MapTransform, and complex dataset handlers like SubjectsDataset.[5] This encapsulation inevitably intertwines the core mathematical image processing logic with physical coordinate metadata (such as affine headers stored in NIfTI format or encapsulated within SimpleITK objects) and global random number generator (RNG) states.[8]

For modern, highly decoupled, data-centric workflows—specifically those targeting pure-function pipeline environments like sciona-atoms-bio and sciona-atoms-dl—there is a critical and growing need to extract the foundational mathematics of these transforms into stateless, dependency-minimized operational units, herein referred to as "atoms." A pure-function atom strictly adheres to the fundamental computer science principle of referential transparency. In this paradigm, given a specific multidimensional array (NDArray) and an explicit set of deterministic parameters—crucially including an isolated and injected RNG seed or state—the function will perpetually yield the exact identical transformed NDArray without mutating any global state, relying on hidden variables, or interacting with the disk.[11]

Architectural Transition: Stateful Pipelines vs. Pure-Function Atoms



Traditional medical imaging frameworks bind tensor data with physical metadata and global state. The pure-function atomic model decouples the mathematical operation, requiring explicit parameter and seed injections to guarantee reproducibility.

This architectural decoupling provides significant operational advantages. When transformations are deeply embedded within Python classes that inherit from complex PyTorch dataset modules, distributing these tasks across multiple worker processes can lead to severe inter-process communication (IPC) bottlenecks, synchronization errors, and non-deterministic behavior resulting from inherited and duplicated global RNG states across processing forks.[6] Extracting these augmentations into pure NumPy and SciPy implementations ensures that the augmentation logic can be executed across CPU threads with zero PyTorch overhead, thereby optimizing dataloader performance prior to GPU transfer.[13]

This comprehensive research report exhaustively investigates the extraction, mathematical formalization, and strict implementation of four best-in-class 3D augmentation atoms: random 3D rotation, 3D elastic deformation, Gaussian noise injection, and volumetric scaling. The analysis is specifically formulated to address the unique domain constraints of two distinct computational stages identified in the project scope: the structural identification of bacterial flagellar motors via cryogenic electron tomography (cryo-ET) in the byu_flagellar_motors_4th/medical_augmentation stage, and the detection of carbonized ink inside X-ray microtomography (CT) scans of ancient papyri in the

vesuvius_ink_detection_1st/volumetric_augmentation stage.[14]

2. Domain-Specific Modality Physics and Constraints

To parameterize augmentation atoms correctly, the physical realities and constraints of the distinct imaging modalities must be thoroughly understood and modeled. The spatial limitations and noise profiles of cryo-ET drastically differ from those of X-ray microtomography. Consequently, the application of universal default parameters across both domains will result in catastrophic structural degradation or insufficient regularization during model training.

2.1 Cryogenic Electron Tomography (Cryo-ET) and Flagellar Motors

Cryo-electron tomography allows for the 3D visualization of cellular ultrastructures and intricate macromolecular complexes in their native, frozen-hydrated state at near-atomic resolutions.[16] In the specific computational challenge of locating bacterial flagellar motors, algorithms must accurately identify a highly symmetric molecular machine composed of distinct functional protein rings. The flagellar motor spans across the two cellular membranes of the bacterium, possessing a rotor (comprising the P-ring and L-ring) that spins the flagellum fiber, and a stationary stator (comprising the C-ring and MS-ring) that provides essential structural stability.[14]

The defining characteristic of cryo-ET is its exceptionally low signal-to-noise ratio (SNR), frequently estimated to be between 0.01 and 0.1.[19] To prevent rapid radiation damage and the melting of the vitreous ice from the electron beam, the total electron dose applied to the biological specimen must be severely restricted, resulting in highly noisy projections.[16] Furthermore, biological specimens are tilted inside the microscope—typically between -60° and +60° with a step size of 1 to 3 degrees—rather than a full 180° rotation.[16] This limited angular acquisition creates a fundamental "missing wedge" of information in Fourier space, which manifests in the reconstructed spatial domain as blurring and elongation artifacts along the Z-axis (the beam axis).[16]

Consequently, augmentations applied to cryo-ET tomograms must prioritize rotational invariance. Because bacterial flagellar motors can insert into the curved bacterial membrane at any arbitrary angle relative to the imaging plane, detection algorithms must be highly robust to full 3D rotations in SO(3).[23] In practice, this means sampling random 3D rotations across all possible axes.

However, due to the extreme noise and the precise geometric arrangement of the ~100-kDa protein subunits within the flagellar motor, heavy elastic deformations must be applied with extreme caution.[24] Elastic deformation simulates organic tissue warping, which is physically unrealistic for rigid molecular nanomachines. Applying strong elastic displacement fields risks irreversibly destroying the fragile structural signal of the motor rings. Therefore, when utilizing elastic deformation in cryo-ET contexts, the standard deviation of the deformation grid (σ) must be kept very large relative to the object to preserve local secondary structures (like alpha

helices), while the maximum displacement magnitude (α) must be kept exceedingly small (e.g., 1-2 voxels) to avoid shearing the molecule apart.[21] Instead of spatial warping, robust Gaussian noise injection, calibrated precisely to the voxel-wise variance of the biological signal, is primarily utilized to simulate varying degrees of experimental exposure and detector noise.[25]

2.2 X-Ray Microtomography (CT) and Vesuvius Surface Volumes

The Vesuvius Challenge presents an entirely different modality. It involves the task of detecting carbonized ink within ancient, petrified papyrus scrolls that were buried by the pyroclastic flows of Mount Vesuvius in 79 C.E..[15] The data provided to algorithms consists of 3D surface volumes extracted from ultra-high-resolution X-ray microtomography scans acquired at particle accelerators.[27] These surface volumes are essentially thick topological sheets—fixed at exactly 65 slices deep in the Z-direction—representing a flattened region of papyrus where ink may reside on the absolute surface or permeate down into the internal plant fibers.[28]

The physics of this problem stands in stark contrast to cryo-ET. The carbon-based ink used in antiquity possesses a radiological density almost identical to the carbonized papyrus substrate itself.[30] Therefore, the target signal relies on highly subtle textual and textural patterns, such as the distinct "crackle patterns" formed as the ink dried and shrank, rather than dense macromolecular geometries.[30]

Because the Vesuvius data is formatted as strictly bounded 65-layer surface volumes (where the Z-axis inherently represents the physical depth into the papyrus sheet), arbitrary isotropic 3D rotation is mathematically and physically invalid. Applying an arbitrary rotation around the X or Y axes would rotate the 2D papyrus sheet out of the Z-plane, blending depth data into spatial data and destroying the surface volume representation entirely.[28] Therefore, 3D rotations for this specific application must be strictly restricted to exactly 90-degree discrete increments around the Z-axis, combined with standard horizontal and vertical 2D flips.[32]

Conversely, because ancient papyrus exhibits highly organic, non-linear warping, stretching, and tearing due to the heat of the volcano and the passage of two millennia, elastic deformation is incredibly effective.[32] Applying random elastic grids teaches the segmentation networks to achieve spatial invariance to ancient morphological damage, significantly reducing overfitting.[32] In this CT domain, the amplitude of the elastic deformation can be much higher than in cryo-ET.

Augmentation Parameter	**Cryo-ET (Flagellar Motors)**	**Micro-CT (Vesuvius Ink)**
Rotation (Angles)	Continuous SO(3) [0°, 360°] across X, Y, Z	Discrete 90° increments around Z-axis only

Elastic Max Displacement (α)	Low: 0.1 to 2.0 voxels	High: 5.0 to 15.0 voxels
Elastic Smoothness (σ)	High: > 30 voxels	Moderate: 10 to 40 voxels
Volumetric Scaling	Tight limits: [0.90, 1.10]	Broader limits: [0.85, 1.25]
Gaussian Noise Formulation	Variance matched to biological signal SNR	Uniform standard deviation [0, 0.05]

3. Mathematical Formalisms and Algorithmic Extraction

Extracting state-of-the-art volumetric augmentations from heavyweight frameworks like TorchIO and MONAI requires a rigorous translation of object-oriented transform histories into fundamental mathematical operators utilizing pure NumPy and SciPy routines.[2] These frameworks execute chains of deferred evaluations; to build an atom, we must evaluate the transformation mathematics directly on the host CPU.

3.1 Multi-Axis 3D Rotation via Inverse Affine Mapping

Rotating a 3D volume by arbitrary angles ($\theta_x, \theta_y, \theta_z$) around multiple axes simultaneously is mathematically represented by the matrix composition of three fundamental rotation matrices. While NumPy provides highly optimized basic 90-degree rotations via np.rot90 (which operates instantaneously by manipulating array strides without requiring spatial interpolation) [36], rotation by arbitrary, non-orthogonal angles demands resampling the discrete voxel grid.

A common pitfall in pure Python augmentation pipelines is attempting to use the standard scipy.ndimage.rotate function. While superficially accessible, this function is intrinsically limited to rotating a 2D plane defined by exactly two axes at a time.[37] Performing sequential passes of scipy.ndimage.rotate to achieve a 3D rotation results in successive interpolation errors, degrading image quality with each pass. To execute a simultaneous 3D rotation without degradation, one must construct a comprehensive 3D affine transformation matrix and apply it in a single interpolation step.[39]

The generation of the 3D direction cosine matrix R from a sequence of angles is elegantly and purely handled by scipy.spatial.transform.Rotation.from_euler, which maps angles into a precise 3x3 rotation matrix.[41] However, applying this matrix to a volumetric voxel grid using scipy.ndimage.affine_transform introduces a critical mathematical nuance regarding the transformation direction. The SciPy library utilizes "pull" (or backward) resampling mechanics rather than "push" resampling.[43]

In pull resampling (utilized by SciPy's geometric transforms), the algorithm fundamentally iterates through the discrete output grid and applies the inverse rotation matrix to map back to a continuous coordinate in the input grid. To maintain a rotation around the geometric center of the volume, the offset vector s must perfectly compensate for the displacement of the origin. Mathematically, this requires supplying the equation for R^{-1} (the inverse rotation matrix) to the function, while the explicit offset s must be calculated as $s = c - R^{-1}c$, where c represents the geometric center coordinate. This precise mapping ensures that an $Output\_Grid[x, y, z]$ maps backward to an $Input\_Grid[x', y', z']$ for spline interpolation without spatial drift.

Because rotation matrices for rigid body transformations are strictly orthogonal, the required inverse matrix R^{-1} is computationally trivial to obtain; it is exactly equal to the transpose R^T.[43] Thus, a pure rotation atom relies on calculating R^T and solving for the offset s, allowing affine_transform to execute the spatial mapping in a single highly optimized C-backend pass. For discrete voxel masks (such as segmentation labels), the order parameter of affine_transform must be set to 0 (nearest-neighbor) to prevent the interpolation of fractional, non-categorical values, while intensity volumes typically utilize order=3 (cubic spline).[37]

3.2 3D Elastic Deformation and Displacement Fields

Elastic deformation was originally popularized by the seminal U-Net architecture for biomedical segmentation to artificially simulate the organic structural variability inherent in biological tissues.[35] Unlike rigid affine transformations that strictly preserve linearity, collinearity, and parallelism across the volume, elastic deformations apply localized, non-linear warping algorithms.[49]

The pure-function mathematical implementation of elastic deformation avoids iterative physical simulations and instead relies on generating a sparse grid of random vectors, smoothing them, and deploying them to non-linearly displace voxel coordinates. The procedure requires four distinct stages:

1. **Grid Generation:** A coarse displacement grid of arbitrary dimensions (e.g., $k \times k \times k$) is initialized with random values sampled from a standard normal distribution $\mathcal{N}(0, 1)$.

This requires an explicit seed injection to maintain functional purity.[50]

2. **Scaling and Smoothing:** These raw vectors are scaled by a global intensity factor α (often referenced in frameworks as max_displacement). In mathematically robust implementations like the elasticdeform library or the backend of MONAI, the grid is then smoothed using a multi-dimensional Gaussian filter parameterized by a standard deviation σ.[49]
3. **Dense Field Interpolation:** The low-resolution coarse grid is upsampled to match the full native resolution of the input image volume utilizing spline interpolation. This results in a dense, continuous displacement field mapping $(\Delta x, \Delta y, \Delta z)$ for every individual voxel.[35]
4. **Coordinate Mapping:** A rigid coordinate grid representing the original volume is generated via np.meshgrid. The dense displacement field is subsequently added to it, perturbing the regular grid. Finally, scipy.ndimage.map_coordinates is invoked to perform the complex interpolation of the original pixel intensities at these new, non-linearly warped continuous coordinates.[50]

Standard Parameterization for Medical Volumes

The behavior of the elastic field is governed completely by the balance of σ and α. The σ parameter dictates the spatial frequency of the elasticity. A low σ results in high-frequency, jittery, and localized distortions, while a high σ creates broad, slow-rolling topological waves. The α parameter governs the absolute amplitude of the shift.

For high-resolution CT and MRI volumes, standard empirical σ values range from 10 to 40, and α (maximum displacement) ranges between 5 to 15 voxels.[11] TorchIO's RandomElasticDeformation implementation defaults to a max_displacement of 7.5 voxels.[55] For deep learning pipelines operating on localized 3D patches (e.g., 64^3 or 128^3 voxel inputs), σ is typically scaled down to 9-13 to guarantee that the deformation wave fits within the constrained patch boundaries. If σ is larger than the patch, the entire patch simply translates uniformly rather than deforming internally.[34]

3.3 Gaussian Noise Injection for Low-SNR Emulation

The injection of additive Gaussian noise acts as a critical regularizer, preventing neural networks from overfitting to high-frequency artifacts and memorizing clean training datasets. In a pure NumPy functional context, the operation is mathematically trivial—calculated strictly as volume + rng.normal(mean, std, volume.shape)—but determining the accurate standard

deviation (std) requires rigid domain calibration.[56]

In clinical and micro-CT applications, such as the Vesuvius Challenge, voxel intensities are intrinsically tied to physical densities (such as Hounsfield Units).[58] The images are typically clipped to specific ranges to remove outliers and normalized to $[-1, 1]$ or $$ floats.[59] In this normalized space, noise is injected with a mean of 0 and a standard deviation sampled uniformly between 0 and 0.05.[56] The noise mimics electronic detector inaccuracies and stochastic photon scatter.[61]

In stark contrast, cryo-ET applications deal with biological targets like flagellar motors that possess low-contrast structural variances easily obliterated by non-calibrated global noise.[18] State-of-the-art synthetic data generation protocols for cryo-ET mandate calculating the specific voxel-wise variance of the biological signal (v_{sig}) within a clean reference volume. To achieve an experimental target Signal-to-Noise Ratio (SNR_{tar}), the variance of the injected Gaussian noise (σ^2) is calculated algorithmically as $\sigma^2 = v_{sig} / SNR_{tar}$.[25] Simulating experimental cryo-ET conditions requires testing and training models at extreme SNR limits, frequently ranging from 0.05 down to an exceptionally noisy 0.01.[19]

3.4 Volumetric Scaling

Volumetric scaling (zooming) simulates innate physical variations in biological anatomy or mitigates inconsistencies in the voxel spacing established during sensor acquisition. In a pure mathematical context, scaling is merely a specialized application of the affine transformation where the transformation matrix is purely diagonal (e.g., $S = \text{diag}([s_x, s_y, s_z])$).

Using scipy.ndimage.zoom allows for highly optimized execution of this scaling, applying multidimensional spline interpolation over the grid.[63] However, the pure-function architectural paradigm enforces a strict contract: the function must return an array of the identical shape as the input. This is a non-negotiable requirement for generating batched tensor inputs in PyTorch without relying on custom, slow dynamic collate functions.

When a volume is scaled up ($s > 1$), the resulting larger array must be centrally cropped back to the original dimensions. When a volume is scaled down ($s < 1$), the resulting smaller array must be zero-padded equally on all sides. This boundary management preserves the coordinate center of the target object. In standard biological imaging, scaling limits are highly restricted to prevent generating anatomically impossible structures, with zoom factors typically constrained between 0.85 and 1.25.[56]

4. Pure Function Augmentation Atom Catalog

The following concepts meticulously define the sampler atoms required for the specific CDG stages. These functions have been rigorously stripped of framework-specific object states, inheriting properties neither from torchio.Transform nor monai.transforms.Transform. They rely strictly on explicit typed arguments and isolated NumPy random number generators. They fulfill all constraints for domain-specific integration into the sciona-atoms-bio and sciona-atoms-dl repositories.

4.1 Atom: 3D Random Rotation

Name: random_rotate_3d

Description: Rotate a 3D volume by specified angles around the X, Y, and Z axes utilizing an inverse affine transformation to maintain strict geometric center alignment and perfectly preserve the input array dimensions without border clipping.

Source: Based on mathematical logic from scipy.ndimage.affine_transform and the direction cosine matrices of scipy.spatial.transform.Rotation

License: BSD (SciPy)

Concept type: sampler

Signature: (volume: NDArray, angles_deg: tuple[float, float, float], order: int) -> NDArray

Pure function boundary: Pure 3D array and explicit floating-point rotation angles provided as inputs, yielding a rotated array as output. Zero disk I/O, no GPU tensor graph allocation, and zero global NumPy RNG states are invoked during execution.

Contracts:

- require: volume.ndim >= 3 (supports an arbitrary leading channel dimension via recursive looping)
- require: order in (0, 1, 2, 3, 4, 5) (strictly 3 for smooth image splines, 0 for categorical label masks)
- ensure: result.shape == volume.shape
  Witness: Given an 8x8x8 volume containing a single nonzero voxel at index (4, 4, 7), a rotation of (0, 90, 0) degrees along the Y-axis physically shifts the voxel to (7, 4, 4) relative to the exact geometric center coordinate (3.5, 3.5, 3.5).
  Dependencies: numpy, scipy.ndimage.affine_transform, scipy.spatial.transform.Rotation
  CDG stages covered: byu_flagellar_motors_4th/medical_augmentation, vesuvius_ink_detection_1st/volumetric_augmentation

4.2 Atom: 3D Elastic Deformation

Name: elastic_deform_3d

Description: Apply a localized, organic, non-linear spatial deformation to a 3D volume by interpolating a sparse random grid of Gaussian-smoothed displacement vectors across the

voxel space.

Source: Derived from the core algorithmic logic of the elasticdeform library and standard scipy.ndimage.map_coordinates implementations originally defined in U-Net literature.

License: MIT

Concept type: sampler

Signature: (volume: NDArray, sigma: float, alpha: float, seed: int, order: int) -> NDArray

Pure function boundary: Requires an explicit seed parameter to instantiate a localized, short-lived np.random.default_rng(seed) instance. It computes the multi-dimensional displacement grid internally, ensuring execution leaves absolutely no trace on the global numpy random state.

Contracts:

- require: volume.ndim >= 3
- require: sigma > 0 (controls spatial deformation frequency and wave breadth)
- require: alpha >= 0 (controls deformation amplitude and maximum displacement limit)
- ensure: result.shape == volume.shape
  Witness: Given an 8x8x8 volume containing a perfectly centered 4x4x4 block of ones. Applying a deformation with alpha=2.0 and sigma=3.0 will smoothly warp the sharp planar faces of the internal block into organic curves without altering the boundary array dimensions.
  Dependencies: numpy, scipy.ndimage.map_coordinates, scipy.ndimage.gaussian_filter
  CDG stages covered: vesuvius_ink_detection_1st/volumetric_augmentation

4.3 Atom: Volumetric Scaling

Name: scale_volume_3d

Description: Isotropically rescale a 3D volume by specific floating-point factors, strictly applying mathematical center-cropping (for s > 1) or symmetrical zero-padding (for s < 1) to enforce dimensional consistency with the input array.

Source: Based on scipy.ndimage.zoom interpolation mechanics.

License: BSD (SciPy)

Concept type: sampler

Signature: (volume: NDArray, scale_factors: tuple[float, float, float], order: int) -> NDArray

Pure function boundary: Relies on pure continuous array input and immutable scaling tuples. Output array maintains exact input bounds. Extrapolated boundary regions generated by scaling down are filled with constant padding (typically 0.0 or the native image minimum).

Contracts:

- require: volume.ndim == 3
- require: all(s > 0 for s in scale_factors)
- ensure: result.shape == volume.shape
  Witness: An 8x8x8 binary array where only the outer boundary voxels are ones. When scaled with a factor of (0.5, 0.5, 0.5), the outer ones will mathematically collapse into a smaller 4x4x4 boundary perfectly nested in the geometric center of the array, completely surrounded by a padding of zeros.
  Dependencies: numpy, scipy.ndimage.zoom
  CDG stages covered: byu_flagellar_motors_4th/medical_augmentation, vesuvius_ink_detection_1st/volumetric_augmentation

4.4 Atom: Gaussian Noise Injection

Name: add_gaussian_noise_3d

Description: Inject additive Gaussian noise strictly calibrated to a provided standard deviation, mathematically simulating varied sensor detector noise and photon exposure limitations.

Source: Pure numpy statistical and array manipulation routines.

License: BSD (NumPy)

Concept type: sampler

Signature: (volume: NDArray, mean: float, std: float, seed: int) -> NDArray

Pure function boundary: Mandates an isolated random seed parameter to generate the deterministic noise tensor via np.random.default_rng(seed).normal(). Does not mutate the input array in place, instead returning a new allocated copy.

Contracts:

- require: std >= 0
- ensure: result.shape == volume.shape
- ensure: result.dtype == volume.dtype (enforced by safe casting and bounds clipping if required by the target domain)
  Witness: An 8x8x8 volume composed of pure zeros. Injecting noise parameterized with mean=0 and std=1 will yield an output array where the computed statistical variance closely approaches 1.0.
  Dependencies: numpy
  CDG stages covered: byu_flagellar_motors_4th/medical_augmentation

5. Research Conclusions and Pipeline Integration

The transition from deeply stateful, class-oriented augmentation frameworks to a library of pure functional atoms directly addresses the primary research constraints of this investigation. Extracting pure NumPy and SciPy implementations from within the complex layers of TorchIO

and MONAI is fundamentally possible. This is achieved by stripping away the metadata management objects—such as MONAI's MetaTensor and TorchIO's Subject dictionaries—and isolating the core calls to scipy.ndimage.affine_transform and scipy.ndimage.map_coordinates.[6] These pure implementations successfully sever all dependencies on PyTorch (torch), enabling rapid CPU execution and mitigating the severe IPC (Inter-Process Communication) memory leak risks prevalent when complex objects are serialized across DataLoader worker threads.[6]

Furthermore, parameterization must be handled programmatically as a modality-specific hyperparameter constraint rather than relying on library-provided empirical defaults. The data demonstrates that standard parameters are wholly incompatible across modalities. The extreme fragility of the cryo-ET missing wedge necessitates extensive, continuous SO(3) rotational augmentation and precise, variance-calibrated statistical noise, while restricting elastic deformation parameters to highly rigid boundaries ($\sigma > 30, \alpha < 2$).[21] Conversely, the thick, carbon-dense surface volumes of the Vesuvius micro-CT scans are heavily optimized through strictly Z-axis restricted rotations and highly aggressive, organic elastic grid distortions ($\sigma \in , \alpha \in $) to simulate ancient papyrus degradation.[32]

Integrating these four strictly typed, pure mathematical atoms—random_rotate_3d, elastic_deform_3d, scale_volume_3d, and add_gaussian_noise_3d—provides a rigorously bounded, highly performant spatial and textural augmentation basis perfectly suited for integration into sciona-atoms-bio and sciona-atoms-dl. By enforcing referential transparency and strictly localized random number generation, engineers can guarantee deterministic pipeline reproducibility across distributed high-performance computing clusters.

Works cited

1. monai.transforms.spatial.array, accessed May 1, 2026, https://monai-dev.readthedocs.io/en/latest/_gen/monai.transforms.spatial.array.html
2. monai.transforms.transforms — MONAI 0.1.0 documentation, accessed May 1, 2026, https://monai.readthedocs.io/en/0.1.0/_modules/monai/transforms/transforms.html
3. TorchIO download | SourceForge.net, accessed May 1, 2026, https://sourceforge.net/projects/torchio.mirror/
4. TorchIO-project/torchio: Medical imaging processing for AI applications. - GitHub, accessed May 1, 2026, https://github.com/TorchIO-project/torchio
5. MONAI/monai/transforms/transform.py at dev · Project-MONAI/MONAI - GitHub, accessed May 1, 2026, https://github.com/Project-MONAI/MONAI/blob/dev/monai/transforms/transform.py
6. Transforms — MONAI 1.5.0 Documentation, accessed May 1, 2026, https://monai.readthedocs.io/en/1.5.0/transforms.html
7. Transforms — MONAI 1.4.0 Documentation, accessed May 1, 2026, https://monai.readthedocs.io/en/1.4.0/transforms.html
8. Data (pymia.data package) - Read the Docs, accessed May 1, 2026,

https://pymia.readthedocs.io/en/latest/pymia.data.html
9. torchio.data.image - · DOKK, accessed May 1, 2026, https://dokk.org/documentation/torchio/v0.19.3/_modules/torchio/data/image/
10. Transforms - TorchIO - smokeshow, accessed May 1, 2026, https://smokeshow.helpmanual.io/001j4q252e0f3x4k0z38/transforms/
11. Reproducibility of the transformation · TorchIO-project torchio · Discussion #1089 - GitHub, accessed May 1, 2026, https://github.com/fepegar/torchio/discussions/1089
12. Reproduce a given transform · Issue #208 · TorchIO-project/torchio - GitHub, accessed May 1, 2026, https://github.com/fepegar/torchio/issues/208
13. GitHub - MIC-DKFZ/batchgenerators: A framework for data augmentation for 2D and 3D image classification and segmentation, accessed May 1, 2026, https://github.com/MIC-DKFZ/batchgenerators
14. BYU - Locating Bacterial Flagellar Motors 2025 - Kaggle, accessed May 1, 2026, https://www.kaggle.com/competitions/byu-locating-bacterial-flagellar-motors-2025/discussion/566137
15. Vesuvius Challenge, accessed May 1, 2026, https://scrollprize.org/
16. Efficient 3D-CTF correction for cryo-electron tomography using NovaCTF improves subtomogram averaging resolution to 3.4 Å - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC5614107/
17. Structure and Assembly of the Proteus mirabilis Flagellar Motor by Cryo-Electron Tomography - MDPI, accessed May 1, 2026, https://www.mdpi.com/1422-0067/24/9/8292
18. Structural insights into flagellar stator–rotor interactions - PMC - NIH, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6663468/
19. Noise-Transfer2Clean: denoising cryo-EM images based on noise modeling and transfer | Bioinformatics | Oxford Academic, accessed May 1, 2026, https://academic.oup.com/bioinformatics/article/38/7/2022/6522116
20. Cryo-Electron Tomography for Structural Characterization of Macromolecular Complexes - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC3254243/
21. HEMNMA-3D: Cryo Electron Tomography Method Based on Normal Mode Analysis to Study Continuous Conformational Variability of Macromolecular Complexes - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8170028/
22. Rapid Synthesis of Cryo-ET Data for Training Deep Learning Models - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10168359/
23. High-throughput cryo-ET structural pattern mining by unsupervised deep iterative subtomogram clustering | PNAS, accessed May 1, 2026, https://www.pnas.org/doi/10.1073/pnas.2213149120
24. High-confidence 3D template matching for cryo-electron tomography - bioRxiv, accessed May 1, 2026, https://www.biorxiv.org/content/10.1101/2023.09.05.556310v1.full-text
25. Towards Foundation Models for Cryo-ET Subtomogram Analysis - arXiv, accessed May 1, 2026, https://arxiv.org/html/2509.24311v2

26. 1st Place Solution - Kaggle Vesuvius Ink Detection - YouTube, accessed May 1, 2026, https://www.youtube.com/watch?v=IWySc8s00P0
27. Vesuvius Challenge 2023 Grand Prize awarded: we can read the first scroll!, accessed May 1, 2026, https://scrollprize.org/grandprize
28. Vesuvius Challenge - Ink Detection - Kaggle Solutions, accessed May 1, 2026, https://kaggle.curtischong.me/competitions/Vesuvius-Challenge---Ink-Detection
29. Vesuvius Challenge - Ink Detection | Kaggle, accessed May 1, 2026, https://www.kaggle.com/competitions/vesuvius-challenge-ink-detection/data
30. Tutorial: Ink Detection - Vesuvius Challenge, accessed May 1, 2026, https://scrollprize.org/tutorial5
31. Volumetric Fast Fourier Convolution for Detecting Ink on the Carbonized Herculaneum Papyri - CVF Open Access, accessed May 1, 2026, https://openaccess.thecvf.com/content/ICCV2023W/e-Heritage/papers/Quattrini_Volumetric_Fast_Fourier_Convolution_for_Detecting_Ink_on_the_Carbonized_ICCVW_2023_paper.pdf
32. 1st place solution | Kaggle, accessed May 1, 2026, https://www.kaggle.com/competitions/vesuvius-challenge-ink-detection/writeups/ryches-1st-place-solution
33. What is elastic transformation in data augmentation? - Milvus, accessed May 1, 2026, https://milvus.io/ai-quick-reference/what-is-elastic-transformation-in-data-augmentation
34. What is the best data augmentation for 3D brain tumor segmentation? - Diva-portal.org, accessed May 1, 2026, https://www.diva-portal.org/smash/get/diva2:1588376/FULLTEXT01.pdf
35. GitHub - gvtulder/elasticdeform: Differentiable elastic deformations for N-dimensional images (Python, SciPy, NumPy, TensorFlow, PyTorch)., accessed May 1, 2026, https://github.com/gvtulder/elasticdeform
36. Understanding NumPy Rotation with numpy.rot90() | by Amit Yadav - Medium, accessed May 1, 2026, https://medium.com/@amit25173/understanding-numpy-rotation-with-numpy-rot90-402e20d5736c
37. rotate — SciPy v1.17.0 Manual, accessed May 1, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.rotate.html
38. How to rotate a 3D image by a random angle in python - Stack Overflow, accessed May 1, 2026, https://stackoverflow.com/questions/43922198/how-to-rotate-a-3d-image-by-a-random-angle-in-python
39. How do I use scipy.ndimage.interpolate to randomly rotate a numpy array in 3d?, accessed May 1, 2026, https://stackoverflow.com/questions/59464346/how-do-i-use-scipy-ndimage-interpolate-to-randomly-rotate-a-numpy-array-in-3d
40. SciPy - Image Transformation - TutorialsPoint, accessed May 1, 2026, https://www.tutorialspoint.com/scipy/scipy_image_transformation.htm
41. Rotation — SciPy v1.17.0 Manual, accessed May 1, 2026,

https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html
42. from_euler — SciPy v1.17.0 Manual, accessed May 1, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.from_euler.html
43. affine_transform — SciPy v1.17.0 Manual, accessed May 1, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.affine_transform.html
44. Clarify docs that affine_transform works 'backwards'? · Issue #9328 · scipy/scipy - GitHub, accessed May 1, 2026, https://github.com/scipy/scipy/issues/9328
45. How can I use scipy.ndimage.interpolation.affine_transform to rotate an image about its centre? - Stack Overflow, accessed May 1, 2026, https://stackoverflow.com/questions/20161175/how-can-i-use-scipy-ndimage-interpolation-affine-transform-to-rotate-an-image-ab
46. Quick Start — eulerangles 1.0.2 documentation, accessed May 1, 2026, https://eulerangles.readthedocs.io/en/latest/usage/quick_start.html
47. elasticdeform - PyPI, accessed May 1, 2026, https://pypi.org/project/elasticdeform/0.3.1/
48. elasticdeform: Elastic deformations for N-dimensional images - Zenodo, accessed May 1, 2026, https://zenodo.org/records/4563172
49. Deformation - elasticdeform documentation, accessed May 1, 2026, https://elasticdeform.readthedocs.io/en/latest/elasticdeform.html
50. Elastic Deformation, Detailed, Explained - Kaggle, accessed May 1, 2026, https://www.kaggle.com/code/hamzamohiuddin/elastic-deformation-detailed-explained
51. Introduction to 3D medical imaging for machine learning: preprocessing and augmentations, accessed May 1, 2026, https://theaisummer.com/medical-image-processing/
52. map_coordinates — SciPy v1.18.0.dev Manual, accessed May 1, 2026, https://scipy.github.io/devdocs/reference/generated/scipy.ndimage.map_coordinates.html
53. How to set up the interpolation problem using ndimage.map_coordinates? - Stack Overflow, accessed May 1, 2026, https://stackoverflow.com/questions/73546347/how-to-set-up-the-interpolation-problem-using-ndimage-map-coordinates
54. Introduction to 3D medical imaging for machine learning: preprocessing and augmentations, accessed May 1, 2026, https://colab.research.google.com/drive/1fyU_YaZUO3B5qVzBJwGoYZ6XVvVLog30?usp=sharing
55. Data augmentation - fastMONAI, accessed May 1, 2026, https://fastmonai.no/vision_augment.html
56. Human-in-the-Loop—A Deep Learning Strategy in Combination with a Patient-Specific Gaussian Mixture Model Leads to the Fast Characterization of Volumetric Ground-Glass Opacity and Consolidation in the Computed Tomography Scans of COVID-19 Patients - MDPI, accessed May 1, 2026,

https://www.mdpi.com/2077-0383/13/17/5231
57. Transforming Coordinates · TorchIO-project torchio · Discussion #661 - GitHub, accessed May 1, 2026, https://github.com/TorchIO-project/torchio/discussions/661
58. Impact of Noise Level on the Accuracy of Automated Measurement of CT Number Linearity on ACR CT and Computational Phantoms - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10440409/
59. Automated 3D segmentation of rotator cuff muscle and fat from longitudinal CT for shoulder arthroplasty evaluation - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12338071/
60. Nonrigid 3D Medical Image Registration and Fusion Based on Deformable Models - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC3652073/
61. Image Variations - SEG.A. - Segmentation of the Aorta - Grand Challenge, accessed May 1, 2026, https://multicenteraorta.grand-challenge.org/ct-image-variations/
62. Elastic Registration Algorithm Based on Three-dimensional Pulmonary MRI in Quantitative Assessment of Severity of Idiopathic Pulmonary Fibrosis - PMC, accessed May 1, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10597429/
63. Image geometric transforms with NumPy and SciPy - PythonInformer, accessed May 1, 2026, https://www.pythoninformer.com/python-libraries/numpy/image-transforms/
64. MONAI/monai/transforms/post/array.py at dev - GitHub, accessed May 1, 2026, https://github.com/Project-MONAI/MONAI/blob/dev/monai/transforms/post/array.py